

Predicting Employment Status from Lifestyle Factors (NHANES)

Research question: Can self-reported alcohol use and physical-activity behaviors predict whether a respondent is employed? We combine three NHANES modules (ALQ, OCQ, PAQ), build a binary classifier, and then test whether the signal is statistically real.

```
In [24]: import pandas as pd
import numpy as np
```

1. Data preparation

We load the three NHANES `.xpt` modules and outer-merge them on `SEQN` (respondent ID) so each respondent is a single row, even if present in only some modules.

```
In [25]: from pathlib import Path
from functools import reduce

modules = ["ALQ_L", "OCQ_L", "PAQ_L"]
dfs = [pd.read_sas(Path("data") / f"{m}.xpt", format="xport", encoding="utf-8") for m in modules]

df = reduce(lambda left, right: pd.merge(left, right, on="SEQN", how="outer"), dfs)

print(df.shape)
df.head()
```

(8501, 21)

```
Out[25]:
```

	SEQN	ALQ111	ALQ121	ALQ130	ALQ142	ALQ270	ALQ280	ALQ151	ALQ170	OCD150	...	O
0	130378.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	1.0	...	
1	130379.0	1.0	2.0	3.0	5.397605e-79	NaN	NaN	2.0	NaN	1.0	...	
2	130380.0	1.0	10.0	1.0	5.397605e-79	NaN	NaN	2.0	NaN	1.0	...	
3	130384.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	1.0	...	
4	130385.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	4.0	...	

5 rows × 21 columns

`pandas.read_sas` decodes the value 0 in XPORT files as a tiny float ($\sim 5.4e-79$); we restore those to a true 0, since the codebook confirms 0 is a valid response.

```
In [26]: # Fix a pandas.read_sas xport quirk: the value 0 is decoded as ~5.4e-79.
# Restore those to 0 across numeric columns (doc.md confirms 0 is a valid code).
num_cols = df.select_dtypes(include="number").columns
df[num_cols] = df[num_cols].mask(df[num_cols].abs() < 1e-12, 0.0)
```

NHANES encodes "Refused"/"Don't know" as runs of 7s and 9s (e.g. 7/9, or 7777/9999 for wider ranges). We detect and convert these to missing so they aren't treated as real values.

```
In [27]: # Remove NHANES "Refused" (repeated 7s) and "Don't know" (repeated 9s) codes.
# For each column the code is the shortest run of 7s / 9s that exceeds every
# real value, e.g. 7/9 for codes 1-4, or 7777/9999 for a 0-1320 range.
def is_run(x, d):
    return x == int(x) and x > 0 and set(str(int(x))) == {d}

def special_codes(s):
    vals = s.dropna().unique()
    real = [v for v in vals if not is_run(v, "7") and not is_run(v, "9")]
    max_real = max(real) if real else 0
    refused = next((int("7" * w) for w in range(1, 10) if int("7" * w) > max_real), None)
    dont_know = next((int("9" * w) for w in range(1, 10) if int("9" * w) > max_real), None)
    return refused, dont_know

for col in df.columns.drop("SEQN"):
    if np.issubdtype(df[col].dtype, np.number):
        refused, dont_know = special_codes(df[col])
        df[col] = df[col].replace({refused: np.nan, dont_know: np.nan})

print(df.shape)
```

(8501, 21)

We recode raw survey codes into interpretable units (yes/no → 1/0, alcohol-frequency codes → drinking days/year, activity frequency×duration → minutes/week), and apply NHANES skip-pattern logic where an upstream "no" implies a structural 0 rather than missing.

```
In [28]: # ---- Recode variables into meaningful units (per doc.md) ----

# Yes/No -> 1/0
for col in ["ALQ111", "ALQ151"]:
    df[col] = df[col].map({1: 1, 2: 0})

# Alcohol frequency code (0-10, same scheme across these four) -> drinking days/year.
# "Nearly every day" = 300; your example's 365/2 collided with code 3. Tune freely.
FREQ_TO_DAYS = {
    0: 0, 1: 365, 2: 300, 3: 3.5 * 52.18, 4: 2 * 52.18, 5: 52.18,
    6: 2.5 * 12, 7: 12, 8: 9, 9: 4.5, 10: 1.5,
}

for col in ["ALQ121", "ALQ142", "ALQ270", "ALQ280"]:
    df[col] = df[col].map(FREQ_TO_DAYS)

# ALQ130 (drinks/occasion) and ALQ170 (binge count, past 30 days) are already
# meaningful counts -> kept as-is and separate.

# Physical activity: frequency x unit x per-session minutes -> minutes/week.
UNIT_PER_WK = {"D": 7.0, "W": 1.0, "M": 12 / 52.1775, "Y": 1 / 52.1775}
def min_per_week(freq, unit, dur):
    mins = df[freq] * df[unit].map(UNIT_PER_WK) * df[dur]
    mins[df[freq] == 0] = 0.0 # never does it -> 0 (unit/duration are skipped)
    return mins
df["MOD_MIN_WK"] = min_per_week("PAD790Q", "PAD790U", "PAD800")
df["VIG_MIN_WK"] = min_per_week("PAD810Q", "PAD810U", "PAD820")
df = df.drop(columns=["PAD790Q", "PAD790U", "PAD800", "PAD810Q", "PAD810U", "PAD820"])
# PAD680 (sedentary min/day) is already clean -> kept.

# Structural zeros: NHANES skip patterns imply 0, not "missing".
```

```
df.loc[df["ALQ111"] == 0, ["ALQ121", "ALQ130", "ALQ142", "ALQ270", "ALQ280", "ALQ170", "ALQ151"]] = 0
df.loc[df["ALQ121"] == 0, ["ALQ130", "ALQ142", "ALQ270", "ALQ280", "ALQ170"]] = 0

print(df.shape)
```

(8501, 17)

We tally remaining missingness per column to judge which variables are usable and how much imputation each needs.

```
In [29]: null_summary = df.isnull().sum().rename("null_count")
null_summary.to_frame()
```

Out[29]:

	null_count
SEQN	0
ALQ111	3025
ALQ121	3026
ALQ130	3038
ALQ142	3045
ALQ270	4754
ALQ280	4755
ALQ151	3048
ALQ170	4749
OCD150	18
OCQ180	4254
OCQ210	7062
OCQ215	4227
OCQ383	5083
PAD680	436
MOD_MIN_WK	415
VIG_MIN_WK	403

A correlation heatmap helps us spot redundant features (e.g. related alcohol-frequency variables) before modeling.

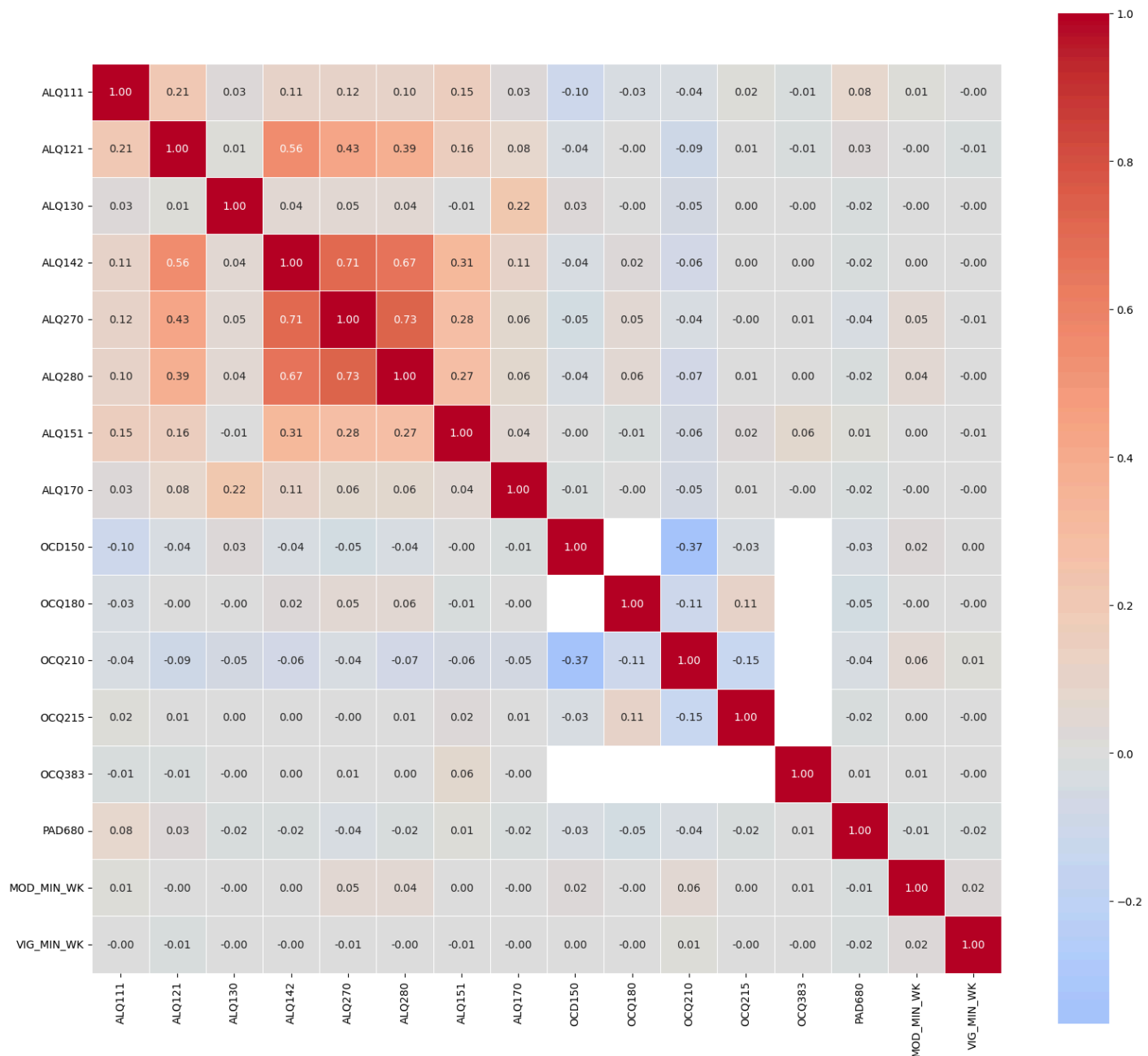
```
In [30]: import seaborn as sns
import matplotlib.pyplot as plt

df_num = df.select_dtypes(include="number").drop(columns="SEQN").round(2)

corr = df_num.corr()

fig, ax = plt.subplots(figsize=(16, 14))
sns.heatmap(corr, annot=True, fmt=".2f", cmap="coolwarm", center=0,
            square=True, linewidths=0.5, ax=ax)
```

```
plt.tight_layout()
plt.show()
```



2. Building the modeling dataset

We define the target from `OCD150` : **employed** = currently working or has a job but absent; **not employed** = everyone else. Critically, we first **exclude retirees** (`OCD150 == 4` with reason `OCQ383 == 3`), since retirement is age-driven and would otherwise make the model detect age rather than employment. The remaining "not employed" class still mixes job-seekers with disabled/student/homemaker respondents, so we read it as "not employed (excl. retired)" rather than strict unemployment. We then drop rows with no employment label and remove leaky/duplicative occupation columns, leaving the numeric lifestyle variables as features `X` .

```
In [31]: from sklearn.model_selection import train_test_split
from sklearn.impute import SimpleImputer

df_ml = df.select_dtypes(include="number").drop(columns="SEQN")

# Exclude retirees: not working (OCD150 == 4) whose reason (OCQ383) is "Retired" (== 3).
```

```

# Retirement is age-driven, so keeping them would make the model detect age, not employment.
# (Code-3 "looking for work" people have OCQ383 = NaN via the skip pattern, so this mask
# only catches code-4 retirees.)
df_ml = df_ml[~((df_ml["OCD150"] == 4) & (df_ml["OCQ383"] == 3))]

# Employed = working (1) or has a job but absent (2); everyone else not working is "not
# employed" (looking for work, disabled, student, homemaker, etc. -- retirees excluded above).
target_map = {1: "employed", 2: "employed", 3: "unemployed", 4: "unemployed"}
df_ml["OCD150"] = df_ml["OCD150"].map(target_map)
df_ml = df_ml.dropna(subset=["OCD150"]).drop(columns=["OCQ210", "OCQ383", "OCQ180", "OCQ215"], err

X = df_ml.drop(columns="OCD150")
y = df_ml["OCD150"]

print(y.value_counts())

```

```

OCD150
employed      4509
unemployed    1986
Name: count, dtype: int64

```

We check multicollinearity with VIF, iteratively dropping any feature with VIF > 5 (protecting PAD680), since collinearity would destabilize the logistic-regression coefficients we interpret later.

```

In [32]: from statsmodels.stats.outliers_influence import variance_inflation_factor

# VIF needs complete data, so work on a median-imputed copy of the features.
# (The real pipeline imputes after the train/test split below.)
X_vif = X.fillna(X.median())

# Report highly correlated pairs for intuition (e.g. ALQ121 / ALQ280).
c = X.corr().abs()
pairs = c.where(np.triu(np.ones(c.shape), k=1).astype(bool)).stack()
print("Correlated pairs (|r| > 0.8):")
print(pairs[pairs > 0.8].sort_values(ascending=False).to_string() or " none")

# Iteratively drop the highest-VIF feature until all <= 5; never drop PAD680.
PROTECTED = {"PAD680"}
features = list(X_vif.columns)
dropped = []
while True:
    vif = pd.Series([variance_inflation_factor(X_vif[features].values, i)
                     for i in range(len(features))], index=features)
    candidates = vif.drop(index=PROTECTED & set(features))
    if candidates.max() <= 5:
        break
    worst = candidates.idxmax()
    dropped.append((worst, round(candidates.max(), 2)))
    features.remove(worst)

print("\nDropped (feature, VIF at removal):", dropped or "none")
print("\nFinal VIF:")
print(vif.sort_values(ascending=False).round(2).to_string())

X = X[features]

```

```
Correlated pairs(|r| > 0.8):  
Series([], )
```

```
Dropped (feature, VIF at removal): none
```

```
Final VIF:
```

```
ALQ111      3.78  
PAD680      3.48  
ALQ142      3.45  
ALQ270      3.00  
ALQ280      2.68  
ALQ121      2.14  
ALQ151      1.30  
ALQ130      1.23  
ALQ170      1.22  
MOD_MIN_WK  1.00  
VIG_MIN_WK  1.00
```

We split 80/20 and fit the median imputer on the training set only, then apply it to the test set, to avoid leaking test information into preprocessing.

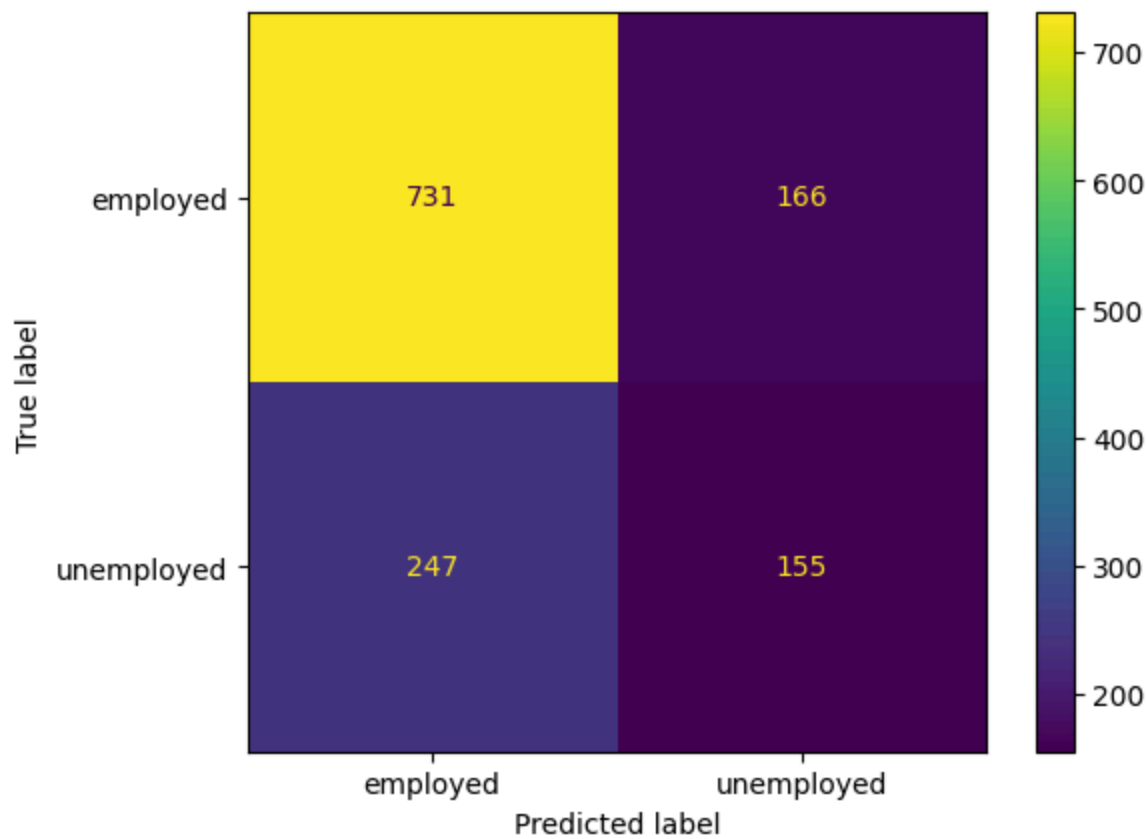
```
In [33]: X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)  
  
imputer = SimpleImputer(strategy="median")  
X_train = pd.DataFrame(imputer.fit_transform(X_train), columns=X.columns)  
X_test = pd.DataFrame(imputer.transform(X_test), columns=X.columns)
```

3. Training and comparing models

We start with a class-balanced Random Forest as a strong, low-tuning baseline and inspect its precision/recall and confusion matrix.

```
In [34]: from sklearn.ensemble import RandomForestClassifier  
from sklearn.metrics import classification_report, ConfusionMatrixDisplay  
  
rf = RandomForestClassifier(class_weight="balanced", random_state=42)  
rf.fit(X_train, y_train)  
  
print(classification_report(y_test, rf.predict(X_test)))  
ConfusionMatrixDisplay.from_estimator(rf, X_test, y_test)  
plt.show()
```

	precision	recall	f1-score	support
employed	0.75	0.81	0.78	897
unemployed	0.48	0.39	0.43	402
accuracy			0.68	1299
macro avg	0.62	0.60	0.60	1299
weighted avg	0.67	0.68	0.67	1299



We read the Random Forest's importances two ways — built-in Gini and permutation importance — since the two can disagree when features are correlated.

```
In [35]: from sklearn.inspection import permutation_importance

gini = pd.Series(rf.feature_importances_, index=X.columns)

perm = permutation_importance(rf, X_test, y_test, random_state=42)
perm = pd.Series(perm.importances_mean, index=X.columns)

importances = pd.DataFrame({"gini": gini, "permutation": perm}).sort_values("permutation", ascending=False)
```

Out[35]:

	gini	permutation
VIG_MIN_WK	0.185380	1.755196e-02
MOD_MIN_WK	0.278853	7.544265e-03
PAD680	0.210344	4.618938e-03
ALQ142	0.045401	1.539646e-03
ALQ280	0.023356	-2.220446e-17
ALQ111	0.007629	-1.385681e-03
ALQ170	0.029522	-1.847575e-03
ALQ151	0.017434	-3.079292e-03
ALQ270	0.034414	-3.849115e-03
ALQ121	0.095667	-4.618938e-03
ALQ130	0.072001	-4.926867e-03

We try KNN, scaling features first because distance-based methods are sensitive to feature scale.

```
In [36]: from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier

scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test)

knn = KNeighborsClassifier()
knn.fit(X_train_s, y_train)

print(classification_report(y_test, knn.predict(X_test_s)))
```

	precision	recall	f1-score	support
employed	0.72	0.82	0.77	897
unemployed	0.43	0.29	0.35	402
accuracy			0.66	1299
macro avg	0.57	0.56	0.56	1299
weighted avg	0.63	0.66	0.64	1299

We fit a logistic regression as an interpretable linear baseline whose coefficients we examine later.

```
In [37]: from sklearn.linear_model import LogisticRegression

lr = LogisticRegression(class_weight="balanced", max_iter=1000)
lr.fit(X_train_s, y_train)

print(classification_report(y_test, lr.predict(X_test_s)))
```


	precision	recall	f1-score	support
employed	0.78	0.38	0.51	897
unemployed	0.35	0.76	0.48	402
accuracy			0.50	1299
macro avg	0.57	0.57	0.50	1299
weighted avg	0.65	0.50	0.50	1299

We add gradient boosting, which often captures nonlinear interactions the other models miss.

```
In [38]: from sklearn.ensemble import HistGradientBoostingClassifier

gb = HistGradientBoostingClassifier(class_weight="balanced", random_state=42)
gb.fit(X_train, y_train)

print(classification_report(y_test, gb.predict(X_test)))
```

	precision	recall	f1-score	support
employed	0.79	0.68	0.73	897
unemployed	0.45	0.60	0.52	402
accuracy			0.65	1299
macro avg	0.62	0.64	0.62	1299
weighted avg	0.69	0.65	0.66	1299

We compare all four models side by side on accuracy and macro-F1, against a majority-class baseline (the no-information rate). A model is only useful if it clears that baseline by a meaningful margin.

```
In [39]: from sklearn.metrics import accuracy_score, f1_score
from sklearn.dummy import DummyClassifier

# Majority-class baseline: always predict the most frequent class. This is the
# no-information rate that the real models must beat to be useful.
dummy = DummyClassifier(strategy="most_frequent").fit(X_train, y_train)

results = {
    "Baseline (majority)": dummy.predict(X_test),
    "RandomForest": rf.predict(X_test),
    "KNN": knn.predict(X_test_s),
    "LogisticRegression": lr.predict(X_test_s),
    "GradientBoosting": gb.predict(X_test),
}

pd.DataFrame({
    name: {
        "accuracy": accuracy_score(y_test, pred),
        "macro_f1": f1_score(y_test, pred, average="macro"),
    }
    for name, pred in results.items()
}).T
```

Out[39]:

	accuracy	macro_f1
Baseline (majority)	0.690531	0.408470
RandomForest	0.682063	0.604251
KNN	0.658968	0.557517
LogisticRegression	0.498845	0.498370
GradientBoosting	0.652810	0.623225

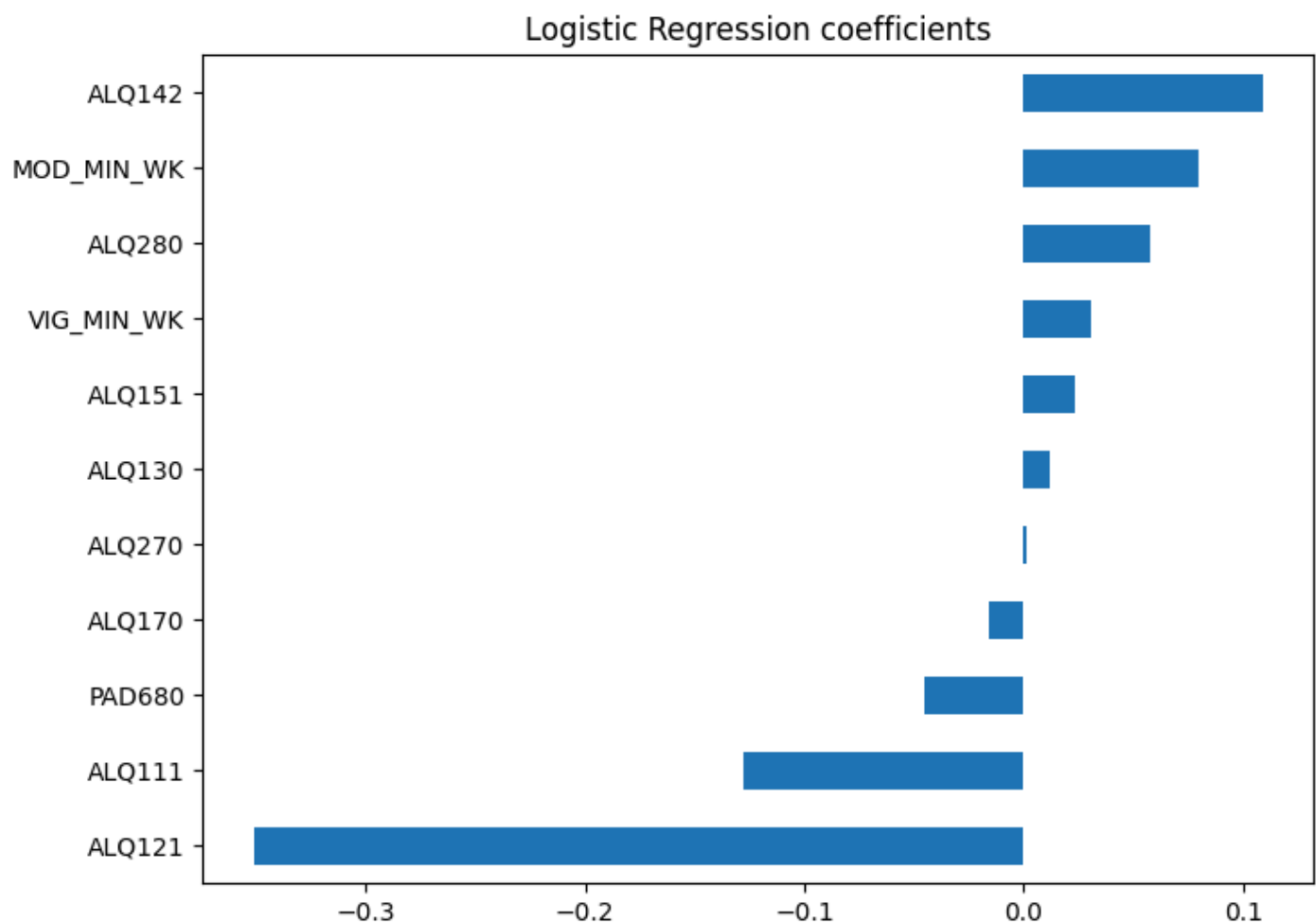
4. Interpreting feature importance

We plot the logistic-regression coefficients to see the direction and magnitude of each feature's effect on the odds of employment.

```
In [40]: # Logistic Regression: signed coefficients (the model itself).
# Log-odds = intercept + sum(coef * feature); positive pushes toward one class.
print("intercept:", lr.intercept_[0])
print("classes:", lr.classes_)

coef = pd.Series(lr.coef_[0], index=X.columns).sort_values()
coef.plot.barh(figsize=(8, 6), title="Logistic Regression coefficients")
plt.show()
```

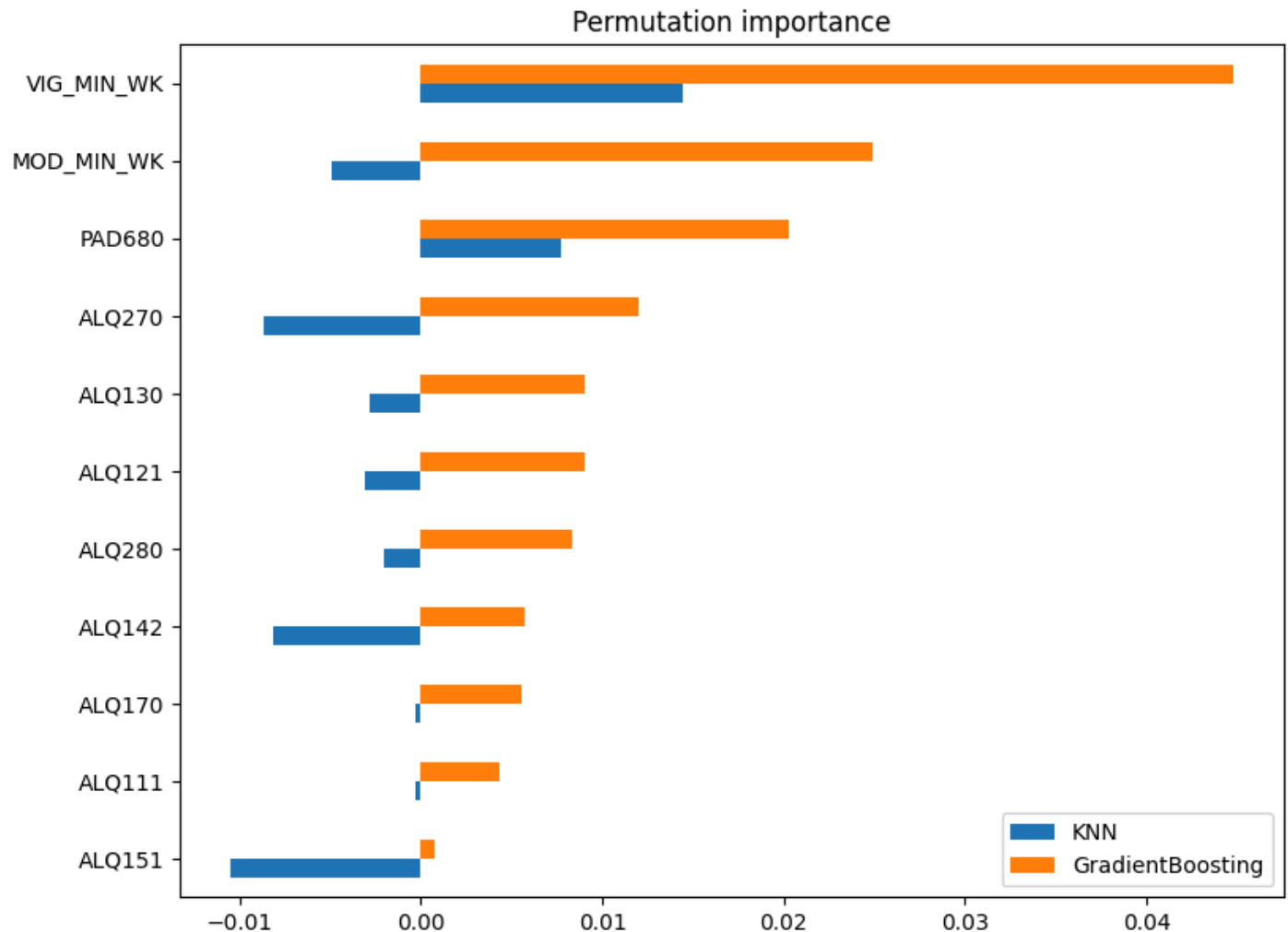
```
intercept: -0.02343583480171029
classes: ['employed' 'unemployed']
```



KNN and gradient boosting lack usable built-in importances, so we use model-agnostic permutation importance for both.

```
In [41]: # KNN and Gradient Boosting have no usable built-in importances,
# so use permutation importance (model-agnostic). KNN uses scaled data.
knn_imp = permutation_importance(knn, X_test_s, y_test, random_state=42).importances_mean
gb_imp = permutation_importance(gb, X_test, y_test, random_state=42).importances_mean

perm_df = pd.DataFrame({"KNN": knn_imp, "GradientBoosting": gb_imp}, index=X.columns)
perm_df.sort_values("GradientBoosting").plot.barh(figsize=(9, 7), title="Permutation importance")
plt.show()
```



We assemble every model's importances into one table; because they're on different scales and signs, we normalize each by its max absolute value and average for an overall ranking.

```
In [42]: imp = pd.DataFrame({
    "RandomForest": rf.feature_importances_,
    "KNN": knn_imp,
    "GradientBoosting": gb_imp,
    "LogisticRegression": lr.coef_[0],
}, index=X.columns)

# Columns are on different scales/signs, so normalize each by its max
# absolute value, then average to rank overall importance.
avg = imp.abs().div(imp.abs().max()).mean(axis=1)
imp["avg_importance"] = avg
imp.sort_values("avg_importance", ascending=False)
```

Out[42]:

	RandomForest	KNN	GradientBoosting	LogisticRegression	avg_importance
VIG_MIN_WK	0.185380	0.014473	0.044804	0.030684	0.688031
MOD_MIN_WK	0.278853	-0.004927	0.024942	0.079653	0.530956
PAD680	0.210344	0.007698	0.020323	-0.045214	0.467131
ALQ121	0.095667	-0.003079	0.009084	-0.351362	0.439647
ALQ142	0.045401	-0.008160	0.005697	0.109511	0.291367
ALQ270	0.034414	-0.008622	0.012009	0.001477	0.247851
ALQ151	0.017434	-0.010470	0.000770	0.023510	0.217504
ALQ130	0.072001	-0.002771	0.009084	0.012071	0.171699
ALQ280	0.023356	-0.002002	0.008314	0.057404	0.142749
ALQ111	0.007629	-0.000308	0.004311	-0.128231	0.127452
ALQ170	0.029522	-0.000308	0.005543	-0.016184	0.074229

We show the same ranking with each model column explicitly normalized, making the averaging step transparent.

In [43]:

```
# Same data, but every model column normalized (|value| / max|value|),
# so avg_importance is simply the row mean of these four columns.
models = ["RandomForest", "KNN", "GradientBoosting", "LogisticRegression"]
norm = imp[models].abs().div(imp[models].abs().max())
norm["avg_importance"] = norm.mean(axis=1)
norm.sort_values("avg_importance", ascending=False)
```

Out[43]:

	RandomForest	KNN	GradientBoosting	LogisticRegression	avg_importance
VIG_MIN_WK	0.664797	1.000000	1.000000	0.087329	0.688031
MOD_MIN_WK	1.000000	0.340426	0.556701	0.226697	0.530956
PAD680	0.754319	0.531915	0.453608	0.128681	0.467131
ALQ121	0.343072	0.212766	0.202749	1.000000	0.439647
ALQ142	0.162814	0.563830	0.127148	0.311677	0.291367
ALQ270	0.123414	0.595745	0.268041	0.004205	0.247851
ALQ151	0.062521	0.723404	0.017182	0.066911	0.217504
ALQ130	0.258204	0.191489	0.202749	0.034354	0.171699
ALQ280	0.083756	0.138298	0.185567	0.163375	0.142749
ALQ111	0.027359	0.021277	0.096220	0.364954	0.127452
ALQ170	0.105868	0.021277	0.123711	0.046061	0.074229

5. Hypothesis testing

Beyond prediction, we ask which associations are statistically significant: we fit a logistic regression, run an omnibus likelihood-ratio test (does *any* feature predict employment?), then per-feature Wald tests with

odds ratios and FDR correction for multiple comparisons.

```
In [44]: import statsmodels.api as sm
from statsmodels.stats.multitest import multipletests

# Outcome as 0/1 (employed = 1), features median-imputed to keep the full sample.
# Note: imputation makes standard errors slightly optimistic vs. complete-case.
y01 = (y == "employed").astype(int)
X_test_hyp = sm.add_constant(X.fillna(X.median()))

result = sm.Logit(y01, X_test_hyp).fit(dis= False)

# 1. Omnibus Likelihood-ratio test: H0 = no feature predicts employment.
print(f"n = {int(result.nobs)}")
print(f"Omnibus LR test: chi2 = {result.llr:.1f}, p = {result.llr_pvalue:.3e}")
print(f"McFadden pseudo-R^2 = {result.prsquared:.3f}")
print("=> reject H0; at least one feature predicts employment"
      if result.llr_pvalue < 0.05 else "=> fail to reject H0")

# 2. Per-feature Wald tests, with odds ratios, CIs, and FDR correction.
ci = result.conf_int()
table = pd.DataFrame({
    "coef": result.params,
    "odds_ratio": np.exp(result.params),
    "or_ci_low": np.exp(ci[0]),
    "or_ci_high": np.exp(ci[1]),
    "p_value": result.pvalues,
}).drop(index="const")
table["p_value_fdr"] = multipletests(table["p_value"], method="fdr_bh")[1]
table["significant"] = table["p_value_fdr"] < 0.05
table.sort_values("p_value").round(4)
```

n = 6495

Omnibus LR test: chi2 = 144.9, p = 1.612e-25

McFadden pseudo-R^2 = 0.018

=> reject H0; at least one feature predicts employment

Out[44]:

	coef	odds_ratio	or_ci_low	or_ci_high	p_value	p_value_fdr	significant
ALQ121	0.0049	1.0049	1.0037	1.0061	0.0000	0.0000	True
ALQ111	0.5394	1.7149	1.3953	2.1078	0.0000	0.0000	True
ALQ142	-0.0025	0.9975	0.9950	1.0000	0.0482	0.1766	False
ALQ151	-0.1440	0.8659	0.7223	1.0380	0.1194	0.2763	False
PAD680	0.0002	1.0002	0.9999	1.0005	0.1366	0.2763	False
MOD_MIN_WK	-0.0000	1.0000	1.0000	1.0000	0.1507	0.2763	False
ALQ280	-0.0018	0.9982	0.9949	1.0016	0.3067	0.4820	False
VIG_MIN_WK	-0.0000	1.0000	1.0000	1.0000	0.3825	0.5259	False
ALQ170	0.0006	1.0006	0.9974	1.0039	0.7081	0.7932	False
ALQ270	-0.0005	0.9995	0.9964	1.0027	0.7739	0.7932	False
ALQ130	-0.0003	0.9997	0.9975	1.0019	0.7932	0.7932	False

Finally, we permutation-test the three non-linear models: by repeatedly shuffling the labels we build a null distribution of scores and check whether each model's real cross-validated performance beats chance.

```
In [45]: from sklearn.model_selection import permutation_test_score, StratifiedKFold, cross_val_score
from sklearn.pipeline import make_pipeline
from IPython.display import display

# Pipelines so imputation (and scaling for KNN) happen inside each CV fold.
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
models = {
    "RandomForest": make_pipeline(
        SimpleImputer(strategy="median"),
        RandomForestClassifier(class_weight="balanced", random_state=42)),
    "KNN": make_pipeline(
        SimpleImputer(strategy="median"), StandardScaler(),
        KNeighborsClassifier()),
    "GradientBoosting": make_pipeline(
        SimpleImputer(strategy="median"),
        HistGradientBoostingClassifier(class_weight="balanced", random_state=42)),
}

# H0: the model predicts no better than chance (labels shuffled).
# permutation_test_score scores one metric (f1_macro); accuracy is added via a
# plain cross_val_score on the same folds (classes are ~balanced, so it's useful too).
perm_results = {}
for name, model in models.items():
    print(f"Running {name}...")
    score, _, pvalue = permutation_test_score(
        model, X, y, scoring="f1_macro", cv=cv,
        n_permutations=100, n_jobs=-1, random_state=42, verbose=0)
    accuracy = cross_val_score(model, X, y, scoring="accuracy", cv=cv, n_jobs=-1).mean()
    perm_results[name] = {"accuracy": accuracy, "f1_macro": score, "p_value": pvalue}

perm_df = pd.DataFrame(perm_results).T
display(perm_df)
```

Running RandomForest...

Running KNN...

Running GradientBoosting...

	accuracy	f1_macro	p_value
RandomForest	0.682833	0.607893	0.009901
KNN	0.666975	0.566103	0.009901
GradientBoosting	0.642494	0.610906	0.009901

6. Sensitivity analysis: does the target definition drive the result?

We re-run the models under two targets to see how much the conclusion depends on how retirees are treated: **(A)** employed vs not-employed, *excluding* retirees (our primary analysis), and **(B)** working vs not working, *including* retirees. If B looks much stronger, that gain reflects reintroduced age/life-stage signal and class rebalancing (the classes shift from $\approx 69/31$ to $\approx 53/47$) — not a genuinely stronger lifestyle effect.

```
In [46]: # Build (X, y) for either target definition straight from the cleaned df.
# The feature set matches the primary analysis (the VIF step dropped nothing).
```

```

def build_xy(drop_retirees: bool):
    d = df.select_dtypes(include="number").drop(columns="SEQN").copy()
    if drop_retirees: # Definition A: drop retirees (OCD150 == 4 & OCQ383 == 3)
        d = d[~((d["OCD150"] == 4) & (d["OCQ383"] == 3))]
    target_map = {1: "working", 2: "working", 3: "not_working", 4: "not_working"}
    d["OCD150"] = d["OCD150"].map(target_map)
    d = d.dropna(subset=["OCD150"]).drop(
        columns=["OCQ210", "OCQ383", "OCQ180", "OCQ215"], errors="ignore")
    return d.drop(columns="OCD150", d["OCD150"])

def make_models(): # same model zoo, each wrapped so impute/scale happen in-fold
    return {
        "RandomForest": make_pipeline(SimpleImputer(strategy="median"),
            RandomForestClassifier(class_weight="balanced", random_state=42)),
        "KNN": make_pipeline(SimpleImputer(strategy="median"), StandardScaler(),
            KNeighborsClassifier()),
        "LogisticRegression": make_pipeline(SimpleImputer(strategy="median"), StandardScaler(),
            LogisticRegression(class_weight="balanced", max_iter=1000)),
        "GradientBoosting": make_pipeline(SimpleImputer(strategy="median"),
            HistGradientBoostingClassifier(class_weight="balanced", random_state=42)),
    }

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
definitions = {"A: excl. retirees": True, "B: incl. retirees": False}

rows = []
for label, drop_ret in definitions.items():
    Xd, yd = build_xy(drop_ret)
    baseline = yd.value_counts(normalize=True).max() # majority-class accuracy
    for name, model in make_models().items():
        acc = cross_val_score(model, Xd, yd, scoring="accuracy", cv=cv, n_jobs=-1).mean()
        f1 = cross_val_score(model, Xd, yd, scoring="f1_macro", cv=cv, n_jobs=-1).mean()
        rows.append({"definition": label, "n": len(yd), "pct_working": (yd == "working").mean(),
            "baseline_acc": baseline, "model": name, "cv_accuracy": acc, "cv_macro_f1":
            f1})

sens = pd.DataFrame(rows)
display(sens.groupby("definition")[["n", "pct_working", "baseline_acc"]].first().round(3))
display(sens.pivot(index="model", columns="definition",
    values=["cv_accuracy", "cv_macro_f1"]).round(3))

```

	n	pct_working	baseline_acc
definition			
A: excl. retirees	6495	0.694	0.694
B: incl. retirees	8483	0.532	0.532

	cv_accuracy		cv_macro_f1	
definition	A: excl. retirees	B: incl. retirees	A: excl. retirees	B: incl. retirees
model				
GradientBoosting	0.642	0.620	0.611	0.620
KNN	0.667	0.574	0.566	0.571
LogisticRegression	0.493	0.545	0.492	0.544
RandomForest	0.682	0.607	0.608	0.605

7. Bonus: can we predict retirement status?

Section 6 showed that adding retirees boosted performance, which suggests retirement itself may be easier to predict from lifestyle behaviors than employment. Here, we flip the target to **retired vs not-retired**, using the same lifestyle features (no occupation columns, so no leakage). We expect this to be the most predictable target so far because retirement is **age-driven** and our features (sedentary time, physical activity, drinking) can be related to age.

```
In [47]: from sklearn.dummy import DummyClassifier

# Target: retired = not working (OCD150 == 4) for reason "Retired" (OCQ383 == 3);
# everyone else with a known work status = not retired. Features stay the lifestyle
# variables only (occupation columns dropped), so there is no label leakage.
d = df.select_dtypes(include="number").drop(columns="SEQN").copy()
d = d[d["OCD150"].notna()]
y_ret = pd.Series(
    np.where((d["OCD150"] == 4) & (d["OCQ383"] == 3), "retired", "not_retired"),
    index=d.index, name="retirement")
X_ret = d.drop(columns=["OCD150", "OCQ210", "OCQ383", "OCQ180", "OCQ215"], errors="ignore")

print(y_ret.value_counts())

# Same model zoo plus a majority-class baseline, scored under the same 5-fold CV.
models_ret = {
    "Baseline (majority)": make_pipeline(
        SimpleImputer(strategy="median"), DummyClassifier(strategy="most_frequent")),
    **make_models(),
}
rows = []
for name, model in models_ret.items():
    acc = cross_val_score(model, X_ret, y_ret, scoring="accuracy", cv=cv, n_jobs=-1).mean()
    f1 = cross_val_score(model, X_ret, y_ret, scoring="f1_macro", cv=cv, n_jobs=-1).mean()
    rows.append({"model": name, "cv_accuracy": acc, "cv_macro_f1": f1})

display(pd.DataFrame(rows).set_index("model").round(3))
```

```
retirement
not_retired    6495
retired        1988
Name: count, dtype: int64
```

	cv_accuracy	cv_macro_f1
model		
Baseline (majority)	0.766	0.434
RandomForest	0.715	0.588
KNN	0.727	0.555
LogisticRegression	0.576	0.504
GradientBoosting	0.648	0.604

```
In [48]: # Is the retirement signal real, and which behaviors drive it?
# (1) Permutation test on the strongest model.
gb_ret = make_pipeline(
    SimpleImputer(strategy="median"),
```



```

HistGradientBoostingClassifier(class_weight="balanced", random_state=42))
score, _, pvalue = permutation_test_score(
    gb_ret, X_ret, y_ret, scoring="f1_macro", cv=cv,
    n_permutations=100, n_jobs=-1, random_state=42)
print(f"GradientBoosting f1_macro = {score:.3f}, permutation p = {pvalue:.4f}")

# (2) Logistic-regression odds ratios: which lifestyle features associate with being retired.
y_ret01 = (y_ret == "retired").astype(int)
Xh = sm.add_constant(X_ret.fillna(X_ret.median()))
res_ret = sm.Logit(y_ret01, Xh).fit(displ=False)
print(f"n = {int(res_ret.nobs)}, "
      f"omnibus p = {res_ret.llr_pvalue:.2e}, McFadden pseudo-R^2 = {res_ret.prsquared:.3f}")

or_table = pd.DataFrame({
    "odds_ratio": np.exp(res_ret.params),
    "p_value": res_ret.pvalues,
}).drop(index="const")
or_table["p_value_fdr"] = multipletests(or_table["p_value"], method="fdr_bh")[1]
or_table["significant"] = or_table["p_value_fdr"] < 0.05
or_table.sort_values("p_value").round(4)

```

GradientBoosting f1_macro = 0.604, permutation p = 0.0099
n = 8483, omnibus p = 2.52e-15, McFadden pseudo-R² = 0.010

Out[48]:

	odds_ratio	p_value	p_value_fdr	significant
ALQ121	1.0025	0.0000	0.0000	True
ALQ130	1.0019	0.0019	0.0103	True
PAD680	0.9996	0.0038	0.0138	True
ALQ111	0.7665	0.0081	0.0222	True
ALQ280	0.9949	0.0368	0.0767	False
ALQ270	0.9963	0.0419	0.0767	False
ALQ142	0.9983	0.1137	0.1787	False
ALQ151	0.8890	0.2006	0.2759	False
ALQ170	0.9994	0.5686	0.6949	False
VIG_MIN_WK	1.0000	0.6518	0.7170	False
MOD_MIN_WK	1.0000	0.9843	0.9843	False

Contrary to our expectation, retirement is **not** more predictable than employment from these lifestyle features. The best model (GradientBoosting) reaches macro-F1 **0.604** — about the same as the employment task (≈ 0.61) — and the logistic regression's pseudo-R² is **0.010**, even *lower* than the employment model's 0.018. No model beats the majority-class accuracy (baseline **0.766**; best is KNN at 0.727), although all the baseline's macro-F1 (0.434) and the permutation test is significant (**p** $\approx$ **0.01**). The conclusion is that our signal is statistically significant but weak.

Takeaway: these behaviors carry only weak information about work/life-stage in general, retirement included. The sensitivity gain in Section 6 therefore came from class rebalancing and a lower baseline, **not** from retirees being easy to flag from lifestyle.

Conclusion

We asked whether self-reported alcohol use and physical-activity behaviors can predict employment status. After excluding retirees (whose non-employment is age-driven rather than labor-market driven), the sample is **4,509 employed vs. 1,986 not employed ($\approx 69\%$ / 31%)**. The result is a **statistically significant but weak correlation**.

Some notes on our findings:

- **Prediction vs. baseline.** Because $\approx 69\%$ of the sample is employed, a majority-class baseline already scores **69.1% accuracy**, and *no model beats it on raw accuracy* (RandomForest comes closest at **68.2%**, GradientBoosting **65.3%**). The models only pull ahead on a population-balanced metric: their **macro-F1 (0.60–0.62)** far exceeds the baseline's **0.41**. In other words, they trade a negligible amount of accuracy to actually identify the minority "not employed" class instead of always guessing "employed".
- **Model comparison. RandomForest has the highest accuracy (0.682)** but a lower macro-F1 (**0.604**), while **GradientBoosting has the highest macro-F1 (0.623)** at lower accuracy (0.653). On this imbalanced target, GradientBoosting's macro-F1 edge makes it the better all-round classifier.
- **Which features.** Statistically, only the **alcohol variables survive FDR correction** — **ALQ121** (past-year drinking frequency, OR ≈ 1.005 per drinking-day) and **ALQ111** (ever drank, OR ≈ 1.71) were significant using Logistic Regression. The ML models rank **physical-activity variables** (vigorous/moderate minutes, sedentary time) highest by importance, so the predictive and inferential views disagree, meaning that no single behavior is a strong predictor.

Main takeaway: lifestyle behaviors are *statistically significantly correlated with, but weak predictors of*, employment status. They explain only a small share of the variation (pseudo- $R^2 \approx 0.02$) and do not beat a trivial majority-class guess on accuracy, so they are not a practical basis for predicting employment. The same holds for retirement, confirming the limitation is the weak information content of these behaviors, not the specific target.

Limitations:

- *Assumed target* — our "not employed" class combines job-seekers with disabled, student, and homemaker respondents, who may not be considered unemployed by other standards.
- *No demographic controls* — age, sex, and income are not in the dataset, so we cannot fully separate lifestyle effects from demographics.
- *No survey weights* — NHANES recommends sample weights (**WTINT2YR**) for population-level estimates; results here are unweighted.
- *Imputation* — median-imputing within the hypothesis-test sample makes standard errors slightly optimistic.